

Міністерство освіти і науки України Національний технічний університет
України «Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4
з дисципліни програмування комп'ютерної графіки
«Моделювання освітлення»

Виконав
Студент групи ІМ-24
Перчук Віталій Олександрович

Київ 2026

Мета роботи: отримати навички програмування освітлення тривимірних об'єктів засобами графіки OpenGL ES.

Завдання.

Потрібно створити у середовищі Android Studio проєкт з ім'ям

Lab4_GLES, зокрема

- написати вихідний текст програми згідно варіанту завдання.

Використати мову Java або Kotlin – на вибір

- Налаштувати програму. Перевірити роботу програми на емуляторі та на фізичному пристрої Android. Варіант

Код:

MainActivity:

```
package com.example.lab4;

import android.content.Context;
import android.opengl.GLES32;
import android.opengl.GLSurfaceView;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.MotionEvent;
import android.view.ScaleGestureDetector;

import androidx.appcompat.app.AppCompatActivity;

import javax.microedition.khronos.egl.EGLConfig;
import javax.microedition.khronos.opengles.GL10;

public class MainActivity extends AppCompatActivity {

    private GLSurfaceView glView;
    private myWorkMode wmRef = null;
    private int currTab = 0;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        glView = new MyGLSurfaceView(this);
        setContentView(glView);

        setTitle("Diffuse lighting");
        myModeStart(new model(), GLSurfaceView.RENDERMODE_WHEN_DIRTY);
        currTab = 1;
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        menu.add(0, 1, 0, "Diffuse lighting");
        menu.add(0, 2, 0, "Specular lighting");
        menu.add(0, 3, 0, "Pyramid");
        menu.add(0, 4, 0, "Nine cube");
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
```

```

        setTitle(item.getTitle());

        switch (item.getItemId()) {
            case 1:
                myModeStart(new model(), GLSurfaceView.RENDERMODE_WHEN_DIRTY);
                currTab = 1;
                return true;

            case 2:
                myModeStart(new mode2(), GLSurfaceView.RENDERMODE_WHEN_DIRTY);
                currTab = 2;
                return true;

            case 3:
                myModeStart(new myFirstMode(), GLSurfaceView.RENDERMODE_CONTINUOUSLY);
                currTab = 3;
                return true;

            case 4:
                myModeStart(new myFirstMode2(), GLSurfaceView.RENDERMODE_CONTINUOUSLY);
                currTab = 4;
                return true;

            default:
                break;
        }

        return super.onOptionsItemSelected(item);
    }

    public void myModeStart(myWorkMode wmode, int rendermode) {
        wmRef = wmode;
        glView.setRenderMode(rendermode);
        glView.requestRender();
    }

    public class MyGLSurfaceView extends GLSurfaceView {

        private final ScaleGestureDetector scaleDetector;
        private float scaleFactor = 1.0f;

        public MyGLSurfaceView(Context context) {
            super(context);
            setEGLContextClientVersion(3);
            setRenderer(new MyGLRenderer());
            setRenderMode(GLSurfaceView.RENDERMODE_WHEN_DIRTY);
            scaleDetector = new ScaleGestureDetector(context, new ScaleListener());
        }

        @Override
        public boolean onTouchEvent(MotionEvent e) {
            if (wmRef == null) return false;
            if (wmRef.onTouchNotUsed()) return false;

            scaleDetector.onTouchEvent(e);

            int cx = this.getWidth();
            int cy = this.getHeight();
            float xtouch = e.getX();
            float ytouch = e.getY();

            switch (e.getActionMasked()) {
                case MotionEvent.ACTION_DOWN:
                    if (wmRef.onActionDown(xtouch, ytouch, cx, cy)) {
                        requestRender();
                    }
                    return true;

                case MotionEvent.ACTION_MOVE:
                    if (!scaleDetector.isInProgress()) {
                        if (wmRef.onActionMove(xtouch, ytouch, cx, cy)) {
                            requestRender();
                        }
                    }
                    return true;
            }
        }
    }

```

```

        return super.onTouchEvent(e);
    }

    private class ScaleListener extends ScaleGestureDetector.SimpleOnScaleGestureListener
    {
        @Override
        public boolean onScale(ScaleGestureDetector detector) {
            scaleFactor = detector.getScaleFactor();
            scaleFactor = Math.max(0.001f, Math.min(scaleFactor, 5.0f));

            if (wmRef != null) {
                if (wmRef.onScale(scaleFactor)) {
                    requestRender();
                }
            }
            return true;
        }
    }

    public class MyGLRenderer implements GLSurfaceView.Renderer {

        private int myRenderHeight = 1;
        private int myRenderWidth = 1;

        @Override
        public void onSurfaceCreated(GL10 unused, EGLConfig config) {
            GLES32.glEnable(GLES32.GL_DEPTH_TEST);
            GLES32.glClearColor(0.0f, 0.0f, 0.0f, 1.0f);

            GLES32.glFrontFace(GLES32.GL_CW);
            GLES32.glEnable(GLES32.GL_CULL_FACE);
            GLES32.glCullFace(GLES32.GL_BACK);
        }

        @Override
        public void onSurfaceChanged(GL10 unused, int width, int height) {
            GLES32.glViewport(0, 0, width, height);
            myRenderWidth = width;
            myRenderHeight = height;
        }

        @Override
        public void onDrawFrame(GL10 unused) {
            GLES32.glClear(GLES32.GL_COLOR_BUFFER_BIT | GLES32.GL_DEPTH_BUFFER_BIT);

            if (wmRef == null) return;

            switch (currTab) {
                case 1:
                case 2:
                    wmRef.setCameraMat();
                    wmRef.setLightMat();

                    if (!wmRef.myCreateShaderProgram(1)) return;
                    wmRef.myCreateScene(1);
                    wmRef.myBindObjBuff(1);
                    wmRef.myUseProgramForDrawing2(myRenderWidth, myRenderHeight);

                    if (!wmRef.myCreateShaderProgram(2)) return;
                    wmRef.getObjMat(1);
                    wmRef.myCreateScene(2);
                    wmRef.myBindObjBuff(2);
                    wmRef.myUseProgramForDrawing(myRenderWidth, myRenderHeight);
                    break;

                case 3:
                    wmRef.setCameraMat();
                    wmRef.setLightMat();

                    if (!wmRef.myCreateShaderProgram(1)) return;
                    wmRef.myCreateScene(1);
                    wmRef.myBindObjBuff(1);
                    wmRef.myUseProgramForDrawing2(myRenderWidth, myRenderHeight);

```

```

        if (!wmRef.myCreateShaderProgram(2)) return;

        wmRef.getObjMat(1);
        wmRef.myCreateScene(2);
        wmRef.myBindObjBuff(2);
        wmRef.myUseProgramForDrawing(myRenderWidth, myRenderHeight);

        wmRef.getObjMat(2);
        wmRef.myCreateScene(3);
        wmRef.myBindObjBuff(2);
        wmRef.myUseProgramForDrawing(myRenderWidth, myRenderHeight);

        wmRef.objMat[6] += 0.5f;
        if (wmRef.objMat[6] >= 360.0f) {
            wmRef.objMat[6] = 0.0f;
        }
        wmRef.setObjMat(2);
        break;

    case 4:
        wmRef.setCameraMat();
        wmRef.setLightMat();

        if (!wmRef.myCreateShaderProgram(1)) return;
        wmRef.myCreateScene(1);
        wmRef.myBindObjBuff(1);
        wmRef.myUseProgramForDrawing2(myRenderWidth, myRenderHeight);

        if (!wmRef.myCreateShaderProgram(2)) return;

        wmRef.getObjMat(1);
        wmRef.myCreateScene(2);
        wmRef.myBindObjBuff(2);

        float baseX = wmRef.objMat[0];
        float baseY = wmRef.objMat[1];
        float baseZ = wmRef.objMat[2];

        float step = 0.35f;

        for (int x = 0; x < 3; x++) {
            for (int y = 0; y < 3; y++) {
                for (int z = 0; z < 3; z++) {

                    wmRef.objMat[0] = baseX + (x - 1) * step;
                    wmRef.objMat[1] = baseY + (y - 1) * step;
                    wmRef.objMat[2] = baseZ + (z - 1) * step;

                    wmRef.myUseProgramForDrawing(myRenderWidth, myRenderHeight);
                }
            }
        }

        // повертаємо назад
        wmRef.objMat[0] = baseX;
        wmRef.objMat[1] = baseY;
        wmRef.objMat[2] = baseZ;

        wmRef.setObjMat(1);
        break;

    default:
        break;
    }
}
}
}
}

```

myWorkMode:

```

package com.example.lab4;

import android.opengl.GLES32;
import android.opengl.Matrix;

```

```

import java.nio.ByteBuffer;
import java.nio.ByteOrder;
import java.nio.FloatBuffer;

import static android.opengl.GLES32.GL_COMPILE_STATUS;

public class myWorkMode {

    protected int gl_Program = 0;
    protected int VAO_id = 0;
    protected int VBO_id = 0;

    protected float[] arrayVertex = null;
    protected int numVertex = 0;

    float[] modelMatrix = new float[16];
    float[] viewMatrix = new float[16];
    float[] projMatrix = new float[16];

    float[] objMat = new float[12];
    float[] cameraMat = new float[9];

    float prevDownX = 0.0f;
    float prevDownY = 0.0f;
    float sensitivity = 0.01f;

    float[] objColor = new float[3];
    float[] lightObj = new float[9];
    float[] distK = new float[3];

    public myWorkMode() {
        gl_Program = 0;
        VAO_id = 0;
        VBO_id = 0;
    }

    public int getProgramId() {
        return gl_Program;
    }

    protected int myCompileShader(int shaderType, String shaderCode) {
        int shaderId = GLES32.glCreateShader(shaderType);
        GLES32.glShaderSource(shaderId, shaderCode);
        GLES32.glCompileShader(shaderId);

        int[] res = new int[1];
        GLES32.glGetShaderiv(shaderId, GL_COMPILE_STATUS, res, 0);
        if (res[0] != 1) return 0;

        return shaderId;
    }

    protected boolean myCompileAndAttachShaders(String vsh, String fsh) {
        gl_Program = -1;

        int vertexShaderId = myCompileShader(GLES32.GL_VERTEX_SHADER, vsh);
        if (vertexShaderId == 0) return false;

        int fragmentShaderId = myCompileShader(GLES32.GL_FRAGMENT_SHADER, fsh);
        if (fragmentShaderId == 0) return false;

        gl_Program = GLES32.glCreateProgram();
        GLES32.glAttachShader(gl_Program, vertexShaderId);
        GLES32.glAttachShader(gl_Program, fragmentShaderId);
        GLES32.glLinkProgram(gl_Program);

        GLES32.glDeleteShader(vertexShaderId);
        GLES32.glDeleteShader(fragmentShaderId);

        if (gl_Program == 0) return false;

        GLES32.glUseProgram(gl_Program);
        return true;
    }
}

```

```

protected void getId_VAO_VBO() {
    int[] tmp = new int[1];

    GLES32.glGenVertexArrays(1, tmp, 0);
    VAO_id = tmp[0];

    GLES32.glGenBuffers(1, tmp, 0);
    VBO_id = tmp[0];
}

protected void myVertexArrayBind2(float[] src, int stride,
                                   String atrib1, int offset1,
                                   String atrib2, int offset2) {

    if (gl_Program <= 0) return;

    ByteBuffer bb = ByteBuffer.allocateDirect(src.length * 4);
    bb.order(ByteOrder.nativeOrder());
    FloatBuffer vertexBuffer = bb.asFloatBuffer();
    vertexBuffer.put(src);
    vertexBuffer.position(0);

    getId_VAO_VBO();

    GLES32.glBindVertexArray(VAO_id);
    GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, VBO_id);
    GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER, src.length * 4, vertexBuffer,
GLES32.GL_STATIC_DRAW);

    int handle = GLES32.glGetAttribLocation(gl_Program, atrib1);
    GLES32.glEnableVertexAttribArray(handle);
    GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride * 4, offset1);

    handle = GLES32.glGetAttribLocation(gl_Program, atrib2);
    GLES32.glEnableVertexAttribArray(handle);
    GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride * 4, offset2);

    GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, 0);
    GLES32.glBindVertexArray(0);
}

protected void myVertexArrayBind1(float[] src, int stride, String atrib1, int offset1) {
    if (gl_Program <= 0) return;

    ByteBuffer bb = ByteBuffer.allocateDirect(src.length * 4);
    bb.order(ByteOrder.nativeOrder());
    FloatBuffer vertexBuffer = bb.asFloatBuffer();
    vertexBuffer.put(src);
    vertexBuffer.position(0);

    getId_VAO_VBO();

    GLES32.glBindVertexArray(VAO_id);
    GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, VBO_id);
    GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER, src.length * 4, vertexBuffer,
GLES32.GL_STATIC_DRAW);

    int handle = GLES32.glGetAttribLocation(gl_Program, atrib1);
    GLES32.glEnableVertexAttribArray(handle);
    GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride * 4, offset1);

    GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, 0);
    GLES32.glBindVertexArray(0);
}

public boolean myCreateShaderProgram(int curr) {
    return false;
}

public void setCameraMat() {}
public void setLightMat() {}
public void getObjMat(int currObj) {}
public void setObjMat(int currObj) {}
public void myBindObjBuff(int curr) {}
protected void myCreateScene(int curr) {}

public void myUseProgramForDrawing(int width, int height) {

```

```

    GLES32.glBindVertexArray(VAO_id);

    Matrix.setIdentityM(modelMatrix, 0);
    Matrix.translateM(modelMatrix, 0, objMat[0], objMat[1], objMat[2]);
    Matrix.rotateM(modelMatrix, 0, objMat[4], objMat[3], 0, 0);
    Matrix.rotateM(modelMatrix, 0, objMat[6], 0, objMat[5], 0);
    Matrix.rotateM(modelMatrix, 0, objMat[8], 0, 0, objMat[7]);
    Matrix.scaleM(modelMatrix, 0, objMat[9], objMat[10], objMat[11]);

    Matrix.setLookAtM(viewMatrix, 0,
        cameraMat[0], cameraMat[1], cameraMat[2],
        cameraMat[3], cameraMat[4], cameraMat[5],
        0, 1, 0);

    float aspect = (float) width / (float) height;
    Matrix.perspectiveM(projMatrix, 0, 90.0f, aspect, 0.1f, 100.0f);

    int handle = GLES32.glGetUniformLocation(gl_Program, "uModelMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, modelMatrix, 0);

    handle = GLES32.glGetUniformLocation(gl_Program, "uViewMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, viewMatrix, 0);

    handle = GLES32.glGetUniformLocation(gl_Program, "uProjMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, projMatrix, 0);

    handle = GLES32.glGetUniformLocation(gl_Program, "vColor");
    GLES32.glUniform3f(handle, objColor[0], objColor[1], objColor[2]);

    handle = GLES32.glGetUniformLocation(gl_Program, "vLightPos");
    GLES32.glUniform3f(handle, lightObj[0], lightObj[1], lightObj[2]);

    handle = GLES32.glGetUniformLocation(gl_Program, "vLightColor");
    GLES32.glUniform3f(handle, lightObj[3], lightObj[4], lightObj[5]);

    handle = GLES32.glGetUniformLocation(gl_Program, "lightK");
    GLES32.glUniform3f(handle, lightObj[6], lightObj[7], lightObj[8]);

    handle = GLES32.glGetUniformLocation(gl_Program, "distK");
    GLES32.glUniform3f(handle, distK[0], distK[1], distK[2]);

    handle = GLES32.glGetUniformLocation(gl_Program, "vEyePos");
    GLES32.glUniform3f(handle, cameraMat[0], cameraMat[1], cameraMat[2]);

    GLES32.glDrawArrays(GLES32.GL_TRIANGLES, 0, numVertex);
    GLES32.glBindVertexArray(0);
}

public void myUseProgramForDrawing2(int width, int height) {
    GLES32.glBindVertexArray(VAO_id);

    Matrix.setIdentityM(modelMatrix, 0);
    Matrix.translateM(modelMatrix, 0, lightObj[0], lightObj[1], lightObj[2]);
    Matrix.scaleM(modelMatrix, 0, 1.0f, 1.0f, 1.0f);

    Matrix.setLookAtM(viewMatrix, 0,
        cameraMat[0], cameraMat[1], cameraMat[2],
        cameraMat[3], cameraMat[4], cameraMat[5],
        0, 1, 0);

    float aspect = (float) width / (float) height;
    Matrix.perspectiveM(projMatrix, 0, 90.0f, aspect, 0.1f, 100.0f);

    int handle = GLES32.glGetUniformLocation(gl_Program, "uModelMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, modelMatrix, 0);

    handle = GLES32.glGetUniformLocation(gl_Program, "uViewMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, viewMatrix, 0);

    handle = GLES32.glGetUniformLocation(gl_Program, "uProjMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, projMatrix, 0);

    handle = GLES32.glGetUniformLocation(gl_Program, "vColor");
    GLES32.glUniform3f(handle, lightObj[3], lightObj[4], lightObj[5]);

    GLES32.glDrawArrays(GLES32.GL_POINTS, 0, numVertex);
}

```



```

        GLES32.glBindVertexArray(0);
    }

    public boolean onTouchNotUsed() {
        return true;
    }

    public boolean onActionDown(float x, float y, int cx, int cy) {
        return false;
    }

    public boolean onActionMove(float x, float y, int cx, int cy) {
        return false;
    }

    public boolean onScale(float scaleFactor) {
        return false;
    }

    public static int addVertexXYZn(float[] vdest, int pos,
                                   float x, float y, float z,
                                   float xn, float yn, float zn) {
        float N = (float) Math.sqrt(xn * xn + yn * yn + zn * zn);
        if (N <= 0.0f) return pos;

        vdest[pos++] = x;
        vdest[pos++] = y;
        vdest[pos++] = z;

        vdest[pos++] = xn / N;
        vdest[pos++] = yn / N;
        vdest[pos++] = zn / N;

        return pos;
    }

    public static int addTriangleXYZn(float[] vdest, int pos,
                                      float x1, float y1, float z1,
                                      float x2, float y2, float z2,
                                      float x3, float y3, float z3) {
        float xn = (z2 - z1) * (y3 - y1) - (y2 - y1) * (z3 - z1);
        float yn = (x2 - x1) * (z3 - z1) - (z2 - z1) * (x3 - x1);
        float zn = (y2 - y1) * (x3 - x1) - (x2 - x1) * (y3 - y1);

        pos = addVertexXYZn(vdest, pos, x1, y1, z1, xn, yn, zn);
        pos = addVertexXYZn(vdest, pos, x2, y2, z2, xn, yn, zn);
        pos = addVertexXYZn(vdest, pos, x3, y3, z3, xn, yn, zn);

        return pos;
    }
}

```

myShaders:

```

package com.example.lab4;

public class myShaders {

    public static final String vertexShaderCode4 =
        "#version 300 es\n" +
        "uniform mat4 uModelMatrix;\n" +
        "uniform mat4 uViewMatrix;\n" +
        "uniform mat4 uProjMatrix;\n" +
        "in vec3 vPosition;\n" +
        "in vec3 vNormal;\n" +
        "out vec3 currentPos;\n" +
        "out vec3 currentNormal;\n" +
        "void main() {\n" +
        "    gl_Position = uProjMatrix * uViewMatrix * uModelMatrix *
vec4(vPosition, 1.0);\n" +
        "    vec4 worldPos = uModelMatrix * vec4(vPosition, 1.0);\n" +
        "    currentPos = worldPos.xyz;\n" +
        "    mat3 normalMat = mat3(transpose(inverse(uModelMatrix)));\n" +
        "    currentNormal = normalize(normalMat * vNormal);\n" +

```

```

        }";

    public static final String fragmentShaderCode4 =
        "#version 300 es\n" +
        "precision mediump float;\n" +
        "in vec3 currentPos;\n" +
        "in vec3 currentNormal;\n" +
        "uniform vec3 vColor;\n" +
        "uniform vec3 vLightColor;\n" +
        "uniform vec3 vLightPos;\n" +
        "uniform vec3 vEyePos;\n" +
        "uniform vec3 distK;\n" +
        "uniform vec3 lightK;\n" +
        "out vec4 resultColor;\n" +

        "void main() {\n" +

        "    vec3 norm = normalize(currentNormal);\n" +
        "    vec3 lightDir = normalize(vLightPos - currentPos);\n" +
        "    float distance = length(vLightPos - currentPos);\n" +
        "    float attenuation = 1.0 / (1.0 + distK.x * distance * distance);\n" +

        "    float diffuse = max(dot(norm, lightDir), 0.0);\n" +

        "    vec3 reflectDir = normalize(reflect(-lightDir, norm));\n" +
        "    vec3 eyeDir = normalize(vEyePos - currentPos);\n" +
        "    float spec = pow(max(dot(eyeDir, reflectDir), 0.0), 64.0);\n" +

        "    vec3 baseColor;\n" +

        "    if (abs(currentPos.y + 0.5) < 0.01) {\n" +
        "        float checker = mod(floor(currentPos.x * 4.0) +\n" +
        "        floor(currentPos.z * 4.0), 2.0);\n" +
        "        if (checker < 1.0)\n" +
        "            baseColor = vec3(0.1, 0.1, 0.1);\n" +
        "        else\n" +
        "            baseColor = vec3(0.9, 0.9, 0.9);\n" +
        "    } else {\n" +
        "        baseColor = vColor; // 🟡 піраміда і куби як раніше\n" +
        "    }\n" +

        "    vec3 vClr = lightK.x * baseColor +\n" +
        "        lightK.y * diffuse * baseColor * vLightColor +\n" +
        "        lightK.z * spec * vLightColor;\n" +

        "    resultColor = vec4(attenuation * vClr, 1.0);\n" +
        "};

    public static final String vertexShaderCode5 =
        "#version 300 es\n" +
        "uniform mat4 uModelMatrix;\n" +
        "uniform mat4 uViewMatrix;\n" +
        "uniform mat4 uProjMatrix;\n" +
        "in vec3 vPosition;\n" +
        "void main() {\n" +
        "    gl_Position = uProjMatrix * uViewMatrix * uModelMatrix *\n" +
        "    vec4(vPosition, 1.0);\n" +
        "    gl_PointSize = 15.0;\n" +
        "};

    public static final String fragmentShaderCode5 =
        "#version 300 es\n" +
        "precision mediump float;\n" +
        "uniform vec3 vColor;\n" +
        "out vec4 outColor;\n" +
        "void main() {\n" +
        "    outColor = vec4(vColor, 1.0);\n" +
        "};
}

```

model1:

```
package com.example.lab4;

public class model extends myWorkMode {

    model() {
        super();
    }

    float[] cameraMat1 = new float[]{
        0.0f, 2.0f, 3.0f,
        0.0f, 0.0f, 0.0f,
        0.0f,
        3.6f,
        0.0f
    };

    public float[] objMat1 = new float[]{
        0.0f, 0.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        3.0f, 1.0f, 3.0f
    };

    float[] lightObj1 = new float[]{
        -0.2f, 0.2f, 0.2f,
        1.0f, 0.2f, 1.0f,
        0.3f, 0.7f, 0.0f
    };

    @Override
    public void setCameraMat() {
        cameraMat = cameraMat1;
    }

    @Override
    public void setLightMat() {
        lightObj = lightObj1;
        distK = new float[]{0.9f, 1.0f, 1.0f};
    }

    @Override
    protected void myCreateScene(int curr) {
        switch (curr) {
            case 1:
                arrayVertex = new float[]{
                    0.0f, 0.0f, 0.0f
                };
                numVertex = 1;
                break;

            case 2:
                arrayVertex = new float[36];
                int pos = 0;

                pos = addTriangleXYZn(arrayVertex, pos,
                    -0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, 0.5f);

                pos = addTriangleXYZn(arrayVertex, pos,
                    -0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, 0.5f,
                    -0.5f, -0.5f, 0.5f);

                objColor = new float[]{1.0f, 1.0f, 1.0f};
                numVertex = 6;
                break;
        }
    }

    @Override
    public void setObjMat(int currObj) {
        objMat1 = objMat.clone();
    }
}
```

```

    }

    @Override
    public void getObjMat(int currObj) {
        objMat = objMat1.clone();
    }

    @Override
    public boolean myCreateShaderProgram(int curr) {
        switch (curr) {
            case 1:
                return myCompileAndAttachShaders(myShaders.vertexShaderCode5,
myShaders.fragmentShaderCode5);
            case 2:
                return myCompileAndAttachShaders(myShaders.vertexShaderCode4,
myShaders.fragmentShaderCode4);
            default:
                return false;
        }
    }

    @Override
    public void myBindObjBuff(int curr) {
        switch (curr) {
            case 1:
                myVertexArrayBind1(arrayVertex, 3, "vPosition", 0);
                break;
            case 2:
                myVertexArrayBind2(arrayVertex, 6, "vPosition", 0, "vNormal", 3 * 4);
                break;
        }
    }

    @Override
    public boolean onTouchNotUsed() {
        return false;
    }

    @Override
    public boolean onActionDown(float x, float y, int cx, int cy) {
        prevDownX = x;
        prevDownY = y;
        return true;
    }

    @Override
    public boolean onActionMove(float x, float y, int cx, int cy) {
        float dx = x - prevDownX;
        float dy = y - prevDownY;

        prevDownX = x;
        prevDownY = y;

        lightObj[0] += dx * sensitivity;
        lightObj[2] += dy * sensitivity;
        return true;
    }

    private final float centerScaleFactor = 1.0f;
    private final float moveSensitivity = 0.1f;

    @Override
    public boolean onScale(float scaleFactor) {
        if (centerScaleFactor > scaleFactor) {
            lightObj[1] -= moveSensitivity;
        } else {
            lightObj[1] += moveSensitivity;
        }
        return true;
    }
}

```

mode2:

```
package com.example.lab4;

public class mode2 extends myWorkMode {

    mode2() {
        super();
    }

    float[] cameraMat1 = new float[]{
        0.0f, 2.0f, 3.0f,
        0.0f, 0.0f, 0.0f,
        0.0f,
        3.6f,
        0.0f
    };

    public float[] objMat1 = new float[]{
        0.0f, 0.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        3.0f, 1.0f, 3.0f
    };

    float[] lightObj1 = new float[]{
        -0.2f, 0.2f, 0.2f,
        1.0f, 0.2f, 1.0f,
        0.3f, 0.0f, 0.7f
    };

    @Override
    public void setCameraMat() {
        cameraMat = cameraMat1;
    }

    @Override
    public void setLightMat() {
        lightObj = lightObj1;
        distK = new float[]{0.0f, 1.0f, 1.0f};
    }

    @Override
    protected void myCreateScene(int curr) {
        switch (curr) {
            case 1:
                arrayVertex = new float[]{
                    0.0f, 0.0f, 0.0f
                };
                numVertex = 1;
                break;

            case 2:
                arrayVertex = new float[36];
                int pos = 0;

                pos = addTriangleXYZn(arrayVertex, pos,
                    -0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, 0.5f);

                pos = addTriangleXYZn(arrayVertex, pos,
                    -0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, 0.5f,
                    -0.5f, -0.5f, 0.5f);

                objColor = new float[]{1.0f, 1.0f, 1.0f};
                numVertex = 6;
                break;
        }
    }

    @Override
    public void setObjMat(int currObj) {
        objMat1 = objMat.clone();
    }
}
```

```

    }

    @Override
    public void getObjMat(int currObj) {
        objMat = objMat1.clone();
    }

    @Override
    public boolean myCreateShaderProgram(int curr) {
        switch (curr) {
            case 1:
                return myCompileAndAttachShaders(myShaders.vertexShaderCode5,
myShaders.fragmentShaderCode5);
            case 2:
                return myCompileAndAttachShaders(myShaders.vertexShaderCode4,
myShaders.fragmentShaderCode4);
            default:
                return false;
        }
    }

    @Override
    public void myBindObjBuff(int curr) {
        switch (curr) {
            case 1:
                myVertexArrayBind1(arrayVertex, 3, "vPosition", 0);
                break;
            case 2:
                myVertexArrayBind2(arrayVertex, 6, "vPosition", 0, "vNormal", 3 * 4);
                break;
        }
    }

    @Override
    public boolean onTouchNotUsed() {
        return false;
    }

    @Override
    public boolean onActionDown(float x, float y, int cx, int cy) {
        prevDownX = x;
        prevDownY = y;
        return true;
    }

    @Override
    public boolean onActionMove(float x, float y, int cx, int cy) {
        float dx = x - prevDownX;
        float dy = y - prevDownY;

        prevDownX = x;
        prevDownY = y;

        lightObj[0] += dx * sensitivity;
        lightObj[2] += dy * sensitivity;
        return true;
    }

    private final float centerScaleFactor = 1.0f;
    private final float moveSensitivity = 0.1f;

    @Override
    public boolean onScale(float scaleFactor) {
        if (centerScaleFactor > scaleFactor) {
            lightObj[1] -= moveSensitivity;
        } else {
            lightObj[1] += moveSensitivity;
        }
        return true;
    }
}

```

myFirstMode:

```
package com.example.lab4;

public class myFirstMode extends myWorkMode {
    myFirstMode() {
        super();
    }

    float[] cameraMat1 = new float[]{
        0.0f, 2.0f, 3.0f,
        0.0f, 0.0f, 0.0f,
        0.0f,
        3.6f,
        0.0f
    };

    public float[] objMat1 = new float[]{
        0.0f, 0.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        3.0f, 1.0f, 3.0f
    };

    public float[] objMat2 = new float[]{
        0.0f, 0.1f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 1.0f, 1.0f
    };

    float[] lightObj1 = new float[]{
        0.7f, 0.2f, 0.0f,
        1.0f, 1.0f, 1.0f,
        0.3f, 0.0f, 0.7f
    };

    @Override
    public void setCameraMat() {
        cameraMat = cameraMat1;
    }

    @Override
    public void setLightMat() {
        lightObj = lightObj1;
        distK = new float[]{
            0.7f, 1.0f, 1.0f
        };
    }

    @Override
    protected void myCreateScene(int curr) {
        switch (curr) {
            case 1:
                arrayVertex = new float[]{
                    0.0f, 0.0f, 0.0f,
                };
                numVertex = 1;
                break;
            case 2:
                arrayVertex = new float[36];
                int pos = 0;
                pos = addTriangleXYZn(
                    arrayVertex, pos,
                    -0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, 0.5f
                );
                pos = addTriangleXYZn(
                    arrayVertex, pos,
                    -0.5f, -0.5f, -0.5f,
                    0.5f, -0.5f, 0.5f,
                    -0.5f, -0.5f, 0.5f
                );
                objColor = new float[]{
                    0.0f, 0.5f, 0.0f
                };
                numVertex = 6;
            }
        }
    }
}
```

```

        break;
    case 3:
        arrayVertex = new float[18 * 6];
        int pos2 = 0;

        pos2 = addTriangleXYZn(
            arrayVertex, pos2,
            -0.5f, -0.5f, 0.5f,
            0.5f, -0.5f, 0.5f,
            0.5f, -0.5f, -0.5f
        );
        pos2 = addTriangleXYZn(
            arrayVertex, pos2,
            -0.5f, -0.5f, 0.5f,
            0.5f, -0.5f, -0.5f,
            -0.5f, -0.5f, -0.5f
        );

        pos2 = addTriangleXYZn(
            arrayVertex, pos2,
            0.0f, 0.5f, 0.0f,
            0.5f, -0.5f, 0.5f,
            -0.5f, -0.5f, 0.5f
        );

        pos2 = addTriangleXYZn(
            arrayVertex, pos2,
            0.0f, 0.5f, 0.0f,
            0.5f, -0.5f, -0.5f,
            0.5f, -0.5f, 0.5f
        );

        pos2 = addTriangleXYZn(
            arrayVertex, pos2,
            0.0f, 0.5f, 0.0f,
            -0.5f, -0.5f, -0.5f,
            0.5f, -0.5f, -0.5f
        );

        pos2 = addTriangleXYZn(
            arrayVertex, pos2,
            0.0f, 0.5f, 0.0f,
            -0.5f, -0.5f, 0.5f,
            -0.5f, -0.5f, -0.5f
        );
        objColor = new float[]{
            0.1f, 0.2f, 0.6f
        };
        numVertex = 18;
        break;
    default:
        break;
}

@Override
public void setObjMat(int currObj) {
    switch (currObj) {
        case 1:
            objMat1 = objMat.clone();
            break;
        case 2:
            objMat2 = objMat.clone();
            break;
        default:
            break;
    }
}

@Override
public void getObjMat(int currObj) {
    switch (currObj) {
        case 1:
            objMat = objMat1.clone();
            break;
        case 2:

```



```

        objMat = objMat2.clone();
        break;
    default:
        break;
    }
}

@Override
public boolean myCreateShaderProgram(int curr) {
    switch (curr) {
        case 1:
            return myCompileAndAttachShaders(myShaders.vertexShaderCode5,
myShaders.fragmentShaderCode5);
        case 2:
            return myCompileAndAttachShaders(myShaders.vertexShaderCode4,
myShaders.fragmentShaderCode4);
        default:
            return false;
    }
}

@Override
public void myBindObjBuff(int curr) {
    switch (curr) {
        case 1:
            myVertexArrayBind1(arrayVertex, 3, "vPosition", 0);
            break;
        case 2:
            myVertexArrayBind2(arrayVertex, 6, "vPosition", 0, "vNormal", 3 * 4);
            break;
        default:
            break;
    }
}

@Override
public boolean onTouchNotUsed() {
    return false;
}

@Override
public boolean onActionDown(float x, float y, int cx, int cy) {
    prevDownX = x;
    prevDownY = y;
    return true;
}

;

@Override
public boolean onActionMove(float x, float y, int cx, int cy) {
    float dx = x - prevDownX;
    float dy = y - prevDownY;
    prevDownX = x;
    prevDownY = y;
    lightObj[0] += dx * sensitivity;
    lightObj[2] += dy * sensitivity;
    return true;
}

;

private void updateCameraOrbitY() {
    float cx = 0.0f;
    float cy = 0.0f;
    float cz = 0.0f;
    float yaw = cameraMat[6];
    float radius = cameraMat[7];
    float pitch = cameraMat[8];
    float x = cx + radius * (float) (Math.cos(pitch) * Math.sin(yaw));
    float y = cy + radius * (float) (Math.sin(pitch));
    float z = cz + radius * (float) (Math.cos(pitch) * Math.cos(yaw));
    cameraMat[0] = x;
    cameraMat[1] = y;
    cameraMat[2] = z;
    cameraMat[3] = cx;
}

```

```

        cameraMat[4] = cy;
        cameraMat[5] = cz;
    }

    private float centerScaleFactor = 1.0f;
    private float moveSensitivity = 0.1f;

    @Override
    public boolean onScale(float scaleFactor) {
        if (centerScaleFactor > scaleFactor) {
            lightObj[1] -= moveSensitivity;
        } else
            lightObj[1] += moveSensitivity;
        return true;
    }
}

```

myFirstMode2:

```

package com.example.lab4;
public class myFirstMode2 extends myWorkMode{
    myFirstMode2() {
        super();
    }
    float[] cameraMat1 = new float[]{
        0.0f, 2.0f, 3.0f,
        0.0f, 0.0f, 0.0f,
        0.0f,
        0.1f,
        0.0f
    };
    public float [] objMat2 = new float[]{
        -0.4f, 0.1f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.1f, 0.1f, 0.1f
    };
    float[] lightObj1 = new float[]{
        0.0f, 2.0f, 3.0f,
        1.0f, 1.0f, 1.0f,
        0.1f, 0.5f, 0.0f
    };
    @Override
    public void setCameraMat(){
        cameraMat = cameraMat1;
    }
    @Override
    public void setLightMat(){
        lightObj = lightObj1;
        distK = new float[]{
            2.0f,1.0f,1.0f
        };
    }
    @Override
    protected void myCreateScene(int curr) {
        switch (curr){
            case 1:
                arrayVertex = new float[]{
                    0.0f, 0.0f, 0.0f,
                };
                numVertex = 1;
                break;
            case 2:
                arrayVertex = new float[36*6];
                int pos2 = 0;
                pos2 = addTriangleXYZn(arrayVertex, pos2,
                    -0.5f, 0.5f, 0.5f,
                    0.5f, 0.5f, 0.5f,
                    0.5f, -0.5f, 0.5f
                );
                pos2 = addTriangleXYZn(arrayVertex, pos2,
                    -0.5f, 0.5f, 0.5f,
                    0.5f, -0.5f, 0.5f,

```

```

        -0.5f, -0.5f, 0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        0.5f, 0.5f, -0.5f,
        -0.5f, 0.5f, -0.5f,
        -0.5f, -0.5f, -0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        0.5f, 0.5f, -0.5f,
        -0.5f, -0.5f, -0.5f,
        0.5f, -0.5f, -0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        0.5f, 0.5f, 0.5f,
        0.5f, 0.5f, -0.5f,
        0.5f, -0.5f, -0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        0.5f, 0.5f, 0.5f,
        0.5f, -0.5f, -0.5f,
        0.5f, -0.5f, 0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        -0.5f, 0.5f, -0.5f,
        -0.5f, 0.5f, 0.5f,
        -0.5f, -0.5f, 0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        -0.5f, 0.5f, -0.5f,
        -0.5f, 0.5f, 0.5f,
        -0.5f, -0.5f, -0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        -0.5f, 0.5f, -0.5f,
        0.5f, 0.5f, -0.5f,
        0.5f, 0.5f, 0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        -0.5f, 0.5f, -0.5f,
        0.5f, 0.5f, 0.5f,
        -0.5f, 0.5f, 0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        -0.5f, -0.5f, 0.5f,
        0.5f, -0.5f, 0.5f,
        0.5f, -0.5f, -0.5f
    );

    pos2 = addTriangleXYZn(arrayVertex, pos2,
        -0.5f, -0.5f, 0.5f,
        0.5f, -0.5f, -0.5f,
        -0.5f, -0.5f, -0.5f
    );

    objColor = new float[]{
        0.0f, 0.9f, 0.8f
    };
    numVertex = 36;
    break;
default:
    break;
}
}

@Override
public void setObjMat(int currObj){
    objMat2 = objMat.clone();
}

@Override
public void getObjMat(int currObj){
    objMat = objMat2.clone();
}

@Override
public boolean myCreateShaderProgram(int curr) {

```

```

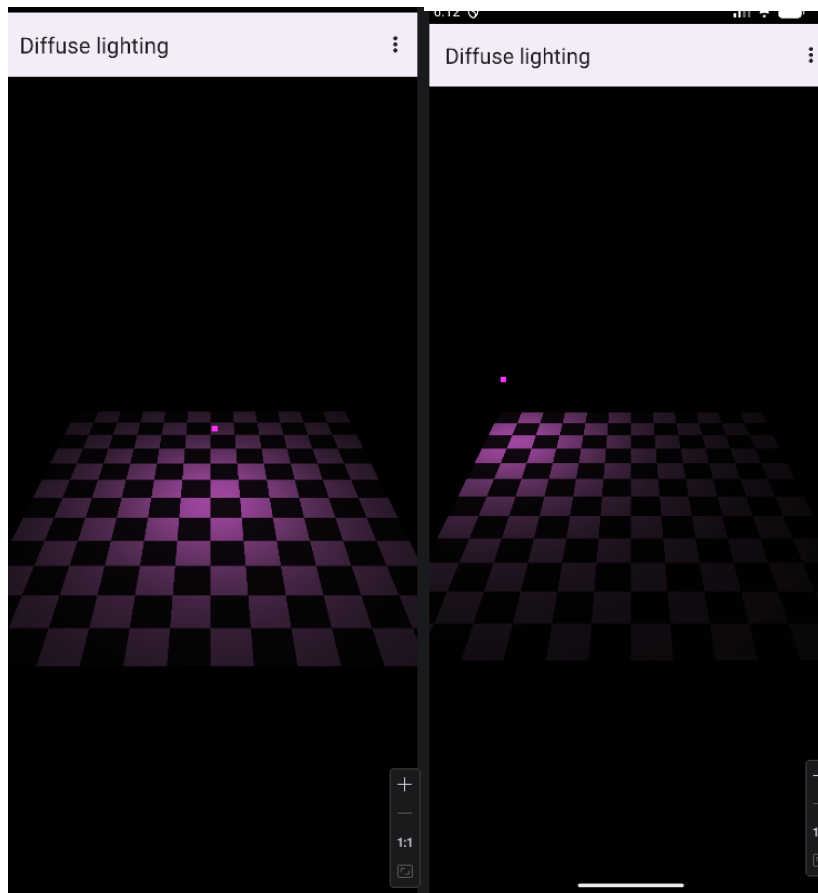
        switch (curr){
            case 1:
                return myCompileAndAttachShaders(myShaders.vertexShaderCode5,
myShaders.fragmentShaderCode5);
            case 2:
                return myCompileAndAttachShaders(myShaders.vertexShaderCode4,
myShaders.fragmentShaderCode4);
            default:
                return false;
        }
    }
    @Override
    public void myBindObjBuff(int curr) {
        switch (curr){
            case 1:
                myVertexArrayBind1(arrayVertex, 3,"vPosition", 0);
                break;
            case 2:
                myVertexArrayBind2(arrayVertex, 6,"vPosition", 0,"vNormal", 3*4);
                break;
            default:
                break;
        }
    }
    @Override
    public boolean onTouchNotUsed() { return false; }
    @Override
    public boolean onActionDown(float x, float y, int cx, int cy){
        prevDownX = x;
        prevDownY = y;
        return true;
    };
    @Override
    public boolean onActionMove(float x, float y, int cx, int cy){
        float dx = x - prevDownX;
        float dy = y - prevDownY;
        prevDownX = x;
        prevDownY = y;
        cameraMat[6] -= dx * sensitivity;
        cameraMat[8] -= dy * sensitivity;
        float limit = 1.5f;
        if (cameraMat[8] > limit) cameraMat[8] = limit;
        if (cameraMat[8] < -limit) cameraMat[8] = -limit;
        updateCameraOrbitY();
        return true;
    };
    private void updateCameraOrbitY() {
        float yaw = cameraMat[6];
        float pitch = cameraMat[8];
        float dx = cameraMat[7] * (float) (Math.cos(pitch) * Math.sin(yaw));
        float dy = cameraMat[7] * (float) (Math.sin(pitch));
        float dz = cameraMat[7] * (float) (Math.cos(pitch) * Math.cos(yaw));
        cameraMat[3] = cameraMat[0] - dx;
        cameraMat[4] = cameraMat[1] + dy;
        cameraMat[5] = cameraMat[2] - dz;
        lightObj[0] = cameraMat[0];
        lightObj[1] = cameraMat[1];
        lightObj[2] = cameraMat[2];
    }
    private float centerScaleFactor = 1.0f;
    private float moveSensitivity = 0.05f;
    @Override
    public boolean onScale(float scaleFactor){

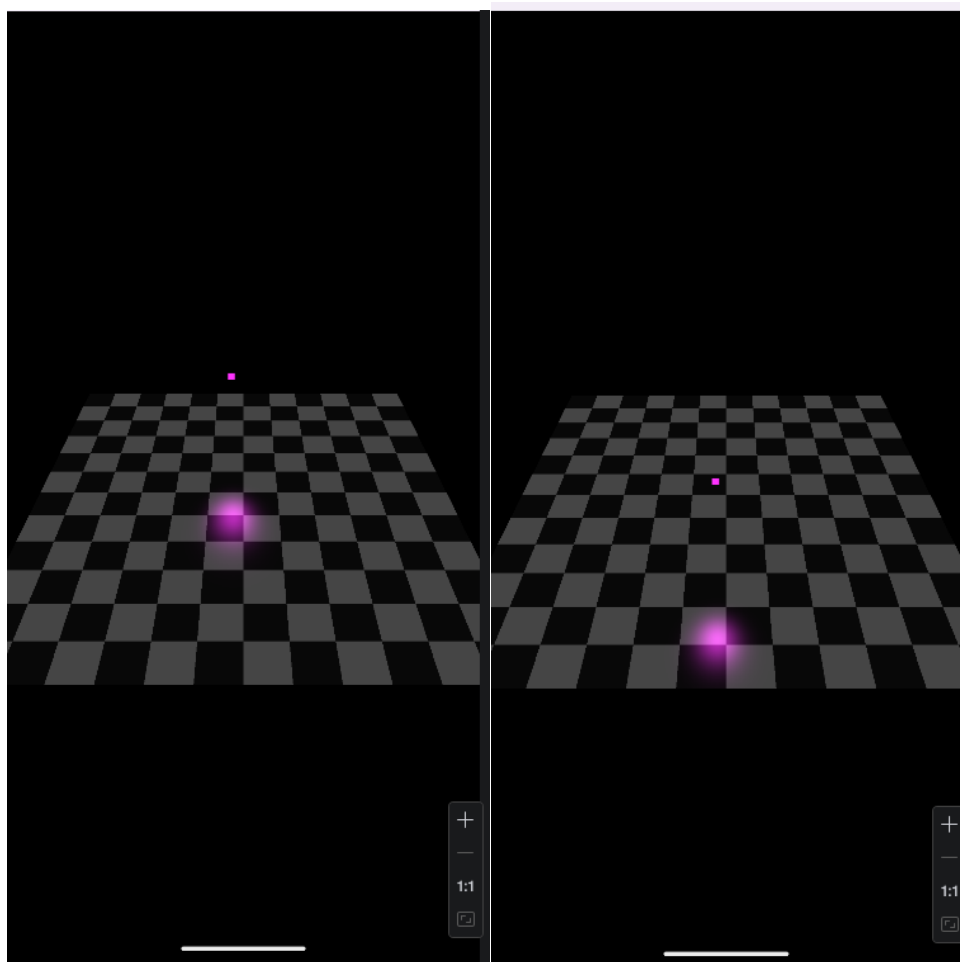
        float dirX = cameraMat[3] - cameraMat[0];
        float dirY = cameraMat[4] - cameraMat[1];
        float dirZ = cameraMat[5] - cameraMat[2];
        float length = (float) Math.sqrt(dirX*dirX + dirY*dirY + dirZ*dirZ);
        dirX /= length;
        dirY /= length;
        dirZ /= length;
        if (centerScaleFactor > scaleFactor){
            cameraMat[0] += dirX * moveSensitivity;
            cameraMat[1] += dirY * moveSensitivity;
            cameraMat[2] += dirZ * moveSensitivity;
        }
    }

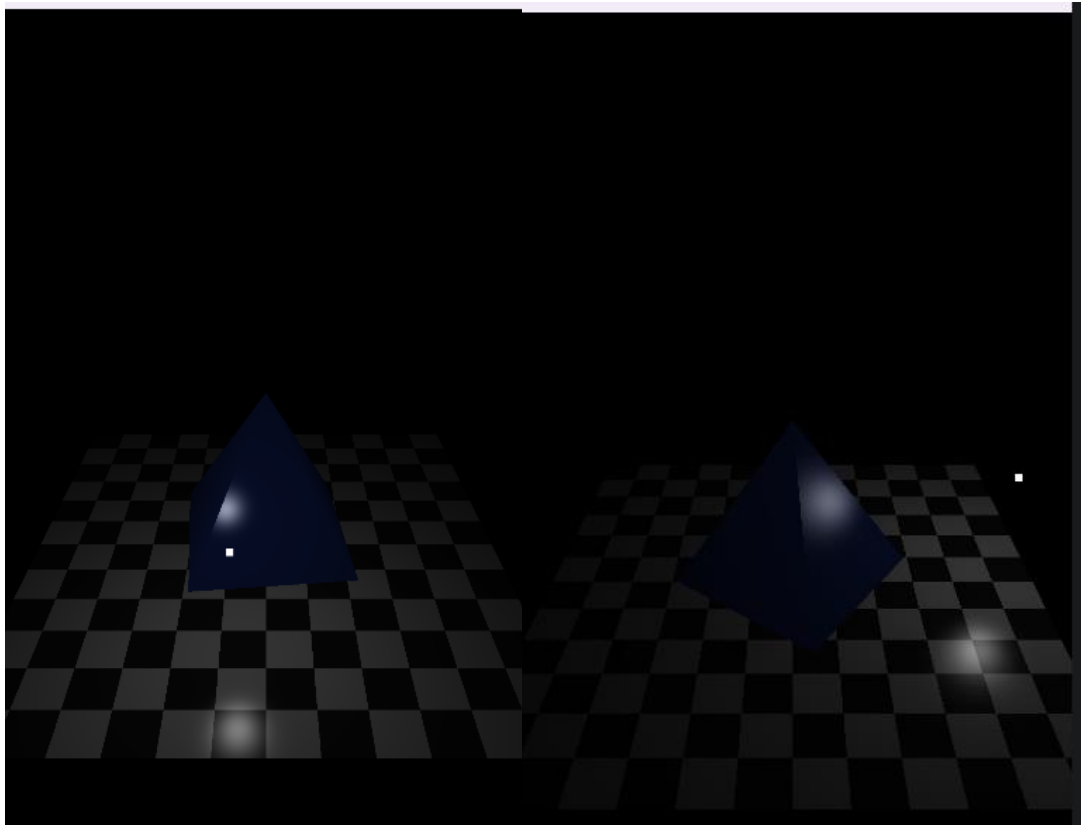
```

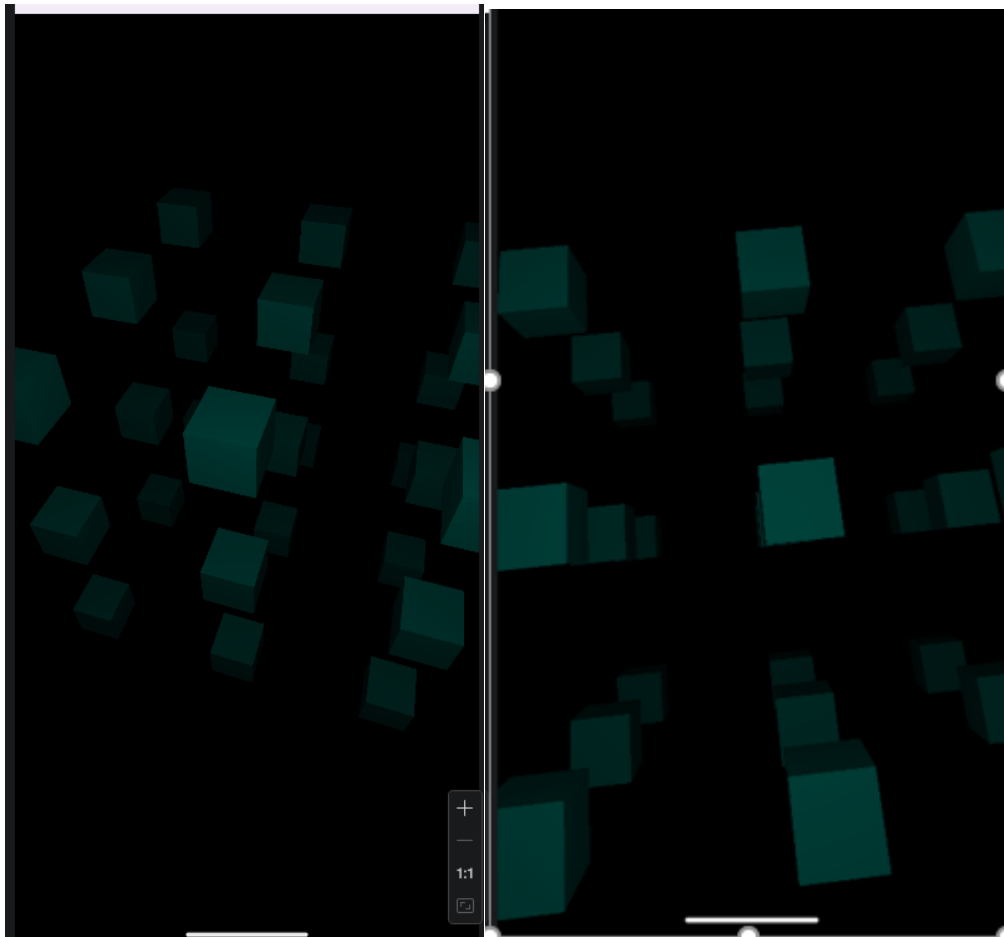
```
    else {  
        cameraMat[0] -= dirX * moveSensitivity;  
        cameraMat[1] -= dirY * moveSensitivity;  
        cameraMat[2] -= dirZ * moveSensitivity;  
    }  
    updateCameraOrbitY();  
    return true;  
}  
}
```

Результат:









Висновок

У результаті виконання лабораторної роботи було отримано практичні навички програмування тривимірної графіки в середовищі Android Studio з використанням OpenGL ES. Зокрема, було реалізовано освітлення об'єктів (дифузне та дзеркальне), використання нормалей, а також керування кольором об'єктів через шейдери. У процесі роботи було створено кілька сцен із різними режимами відображення, включаючи побудову піраміди, множини кубів та взаємодію зі сценою за допомогою користувацького вводу.

Крім того, було засвоєно принципи роботи з матрицями перетворень (моделі, камери та проєкції), організацію структури проєкту та використання шейдерних програм для формування зображення. Отримані знання дозволяють глибше зрозуміти процес рендерингу тривимірної сцени та можуть бути використані для подальшої розробки графічних застосунків.